
Priorização de tickets de suporte técnico com lógica fuzzy

Gabriel Albuquerque, Davi Maciel e Alberto Acosta

CESUPA - Inteligência Artificial e Computacional

Relatório técnico em formato acadêmico inspirado no padrão SBC

Resumo

Este trabalho apresenta um protótipo mínimo e reprodutível de priorização de tickets. A solução usa um sistema fuzzy Mamdani com três entradas, uma saída, doze regras, visualização das funções de pertinência, cenários de teste e análise de sensibilidade.

Palavras-chave

Lógica fuzzy; Mamdani; priorização; suporte técnico; protótipo acadêmico.

GitHub

<https://github.com/Gaalbu/Fuzzy-sistem>

1. Introdução e escopo

Problema e justificativa

O protótipo define a prioridade de atendimento de tickets de suporte técnico acadêmico quando as informações disponíveis são parcialmente subjetivas. A lógica fuzzy é adequada porque transforma avaliações graduais, como impacto baixo, urgência média e tempo longo, em uma decisão numérica interpretável.

Modalidade e requisitos atendidos

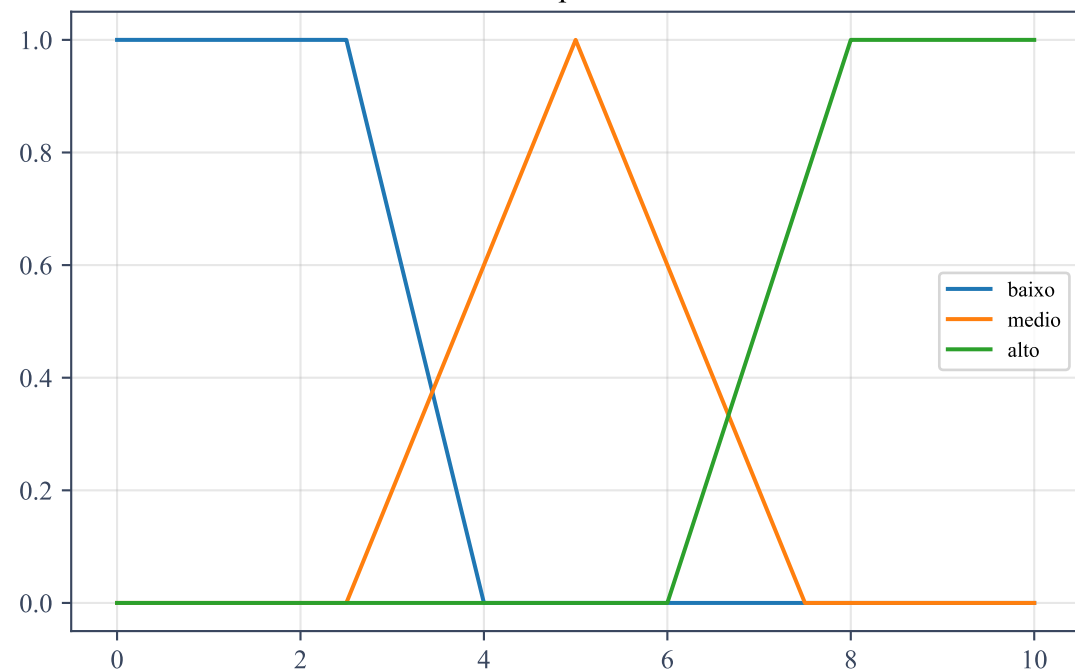
Modalidade escolhida: Opção B, aplicação/produto baseado em controle fuzzy. O modelo possui 3 entradas, 1 saída, 12 regras efetivas, 6 cenários de teste, funções de pertinência visualizadas, manual de execução, código-fonte e declaração de uso de IA.

Extensão opcional

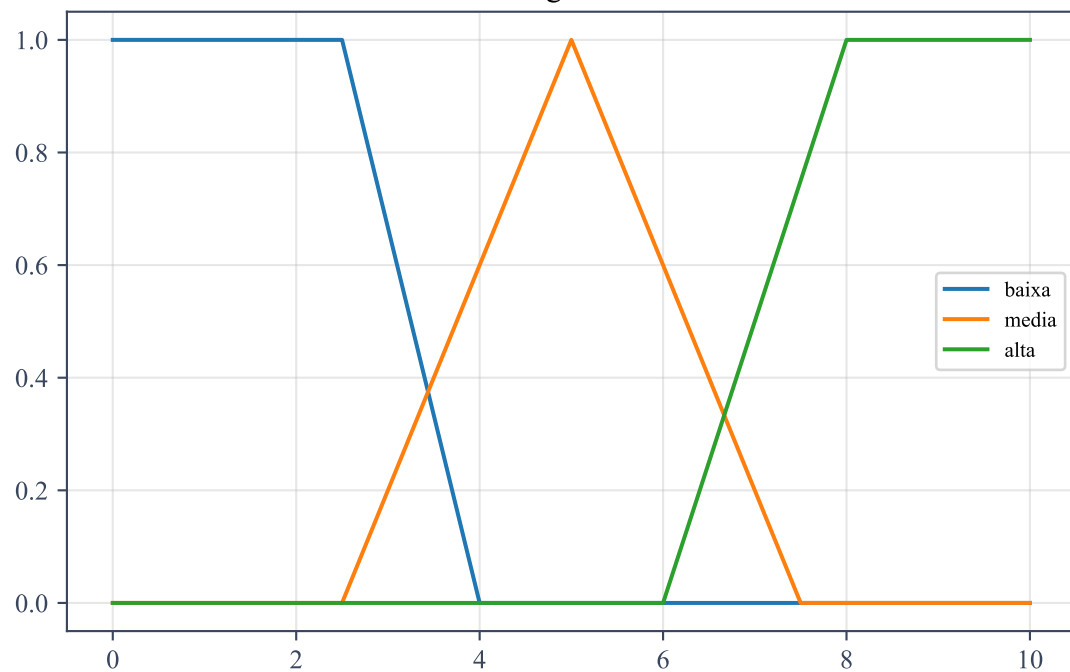
Foi adotado o caminho mais simples da pontuação extra: levantamento bibliográfico complementar. Bases/termos de busca: Google Scholar, IEEE Xplore e Scopus; fuzzy control, Mamdani, TSK e fuzzy rule base. Referências-base: Zadeh (1965), Mamdani & Assilian (1975), Ross (2010) e Klir & Yuan (1995).

2. Funções de pertinência

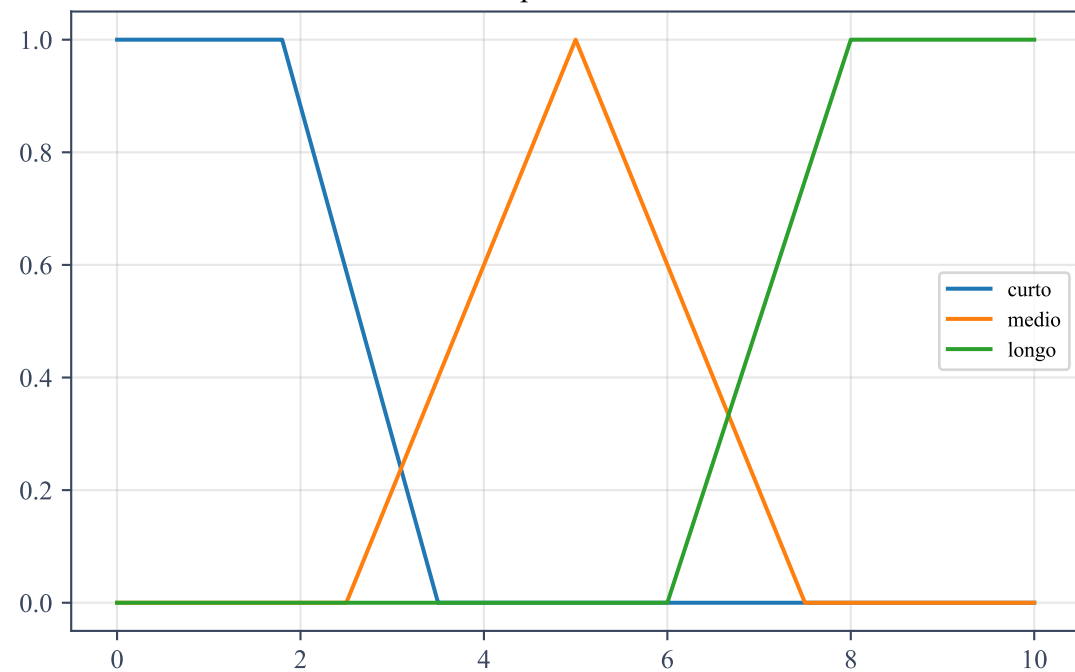
Impacto



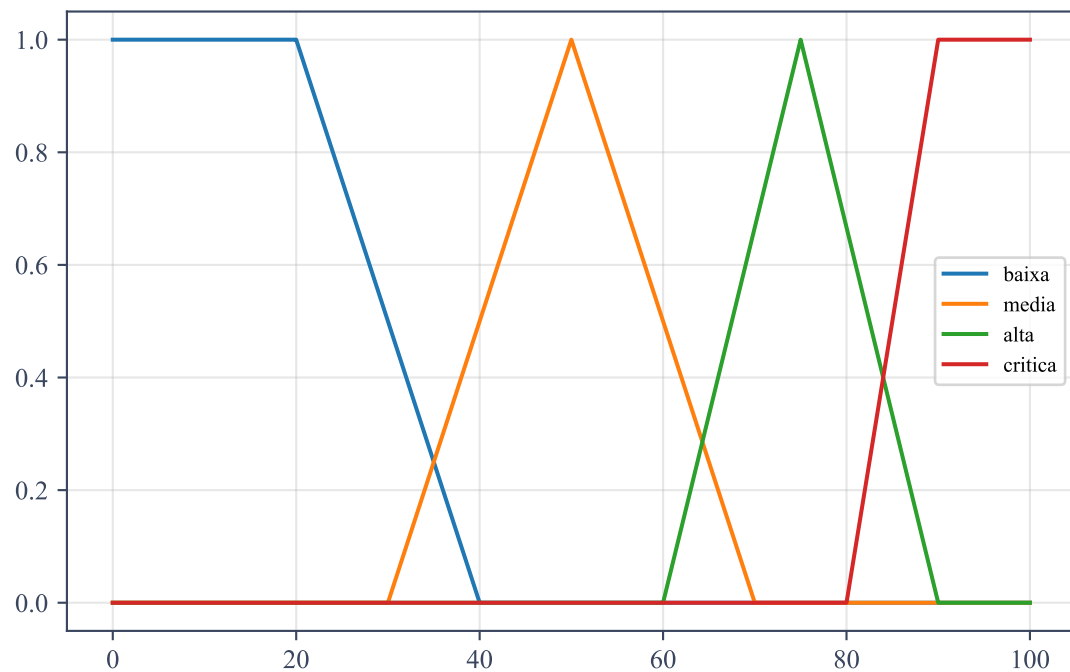
Urgência



Tempo em aberto



Prioridade



3. Base de regras

#	Impacto	Urgência	Tempo	Saída
1	alto	alta	longo	critica
2	alto	alta	curto	alta
3	alto	media	longo	alta
4	alto	baixa	longo	alta
5	medio	alta	longo	alta
6	medio	alta	medio	alta
7	medio	media	medio	media
8	medio	baixa	curto	media
9	baixo	alta	longo	media
10	baixo	media	medio	media
11	baixo	baixa	longo	baixa
12	baixo	baixa	curto	baixa

Inferência: operador AND = mínimo; agregação = máximo; defuzzificação = centróide.

4. Implementação e reprodutibilidade

Arquitetura

O sistema foi mantido em um módulo principal, com um script separado apenas para gerar os artefatos. Essa decisão reduz dependências, facilita a leitura e deixa a execução reproduzível.

Estrutura

src/fuzzy_system.py: implementação do modelo. src/build_artifacts.py: geração dos PDFs e figuras. docs/manual_execucao.md: instruções de execução. docs/declaracao_ia.md: declaração de uso de IA. artifacts/: relatório, apresentação e imagens.

Dependências

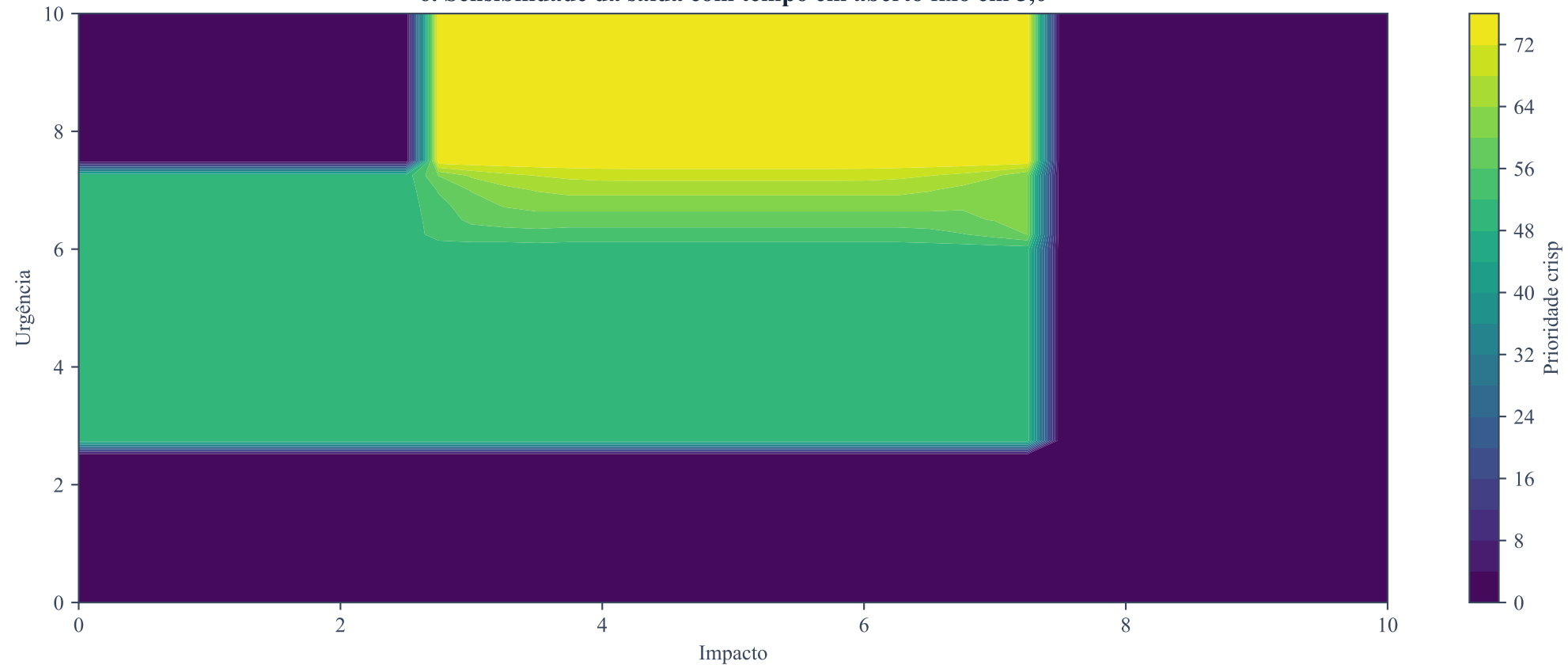
A solução usa apenas Python, numpy e matplotlib, evitando bibliotecas fuzzy prontas para preservar transparência sobre fuzzificação, regras, agregação e defuzzificação.

5. Experimentos

Impacto	Urgência	Tempo	Saída	Classe	Esperado
1.0	1.0	1.0	15.56	baixa	baixa
3.0	4.0	3.0	50.00	media	media
5.0	5.0	5.0	50.00	media	media
8.5	8.5	9.5	92.22	critica	critica
9.0	2.0	7.5	75.00	alta	alta
2.5	9.0	8.0	50.00	media	media

Os cenários cobrem casos baixos, médios, altos, fronteiriços e conflitantes. Os resultados mantêm coerência qualitativa: baixa prioridade em casos simples, prioridade alta em conflito de alto impacto e criticidade quando impacto, urgência e tempo são elevados.

6. Sensibilidade da saída com tempo em aberto fixo em 5,0



7. Conclusão, IA e referências

Conclusão

O protótipo atende ao escopo acadêmico, gera saída contínua, é reprodutível e permite defesa técnica. Como limitações, a base de regras pode ser ampliada e a validação pode ser aprofundada com dados reais.

Declaração de uso de IA

GPT-5.5 da OpenAI foi usado como apoio na escrita, organização e revisão do código. O material final foi conferido manualmente antes da exportação.

Referências

Zadeh (1965); Mamdani & Assilian (1975); Ross (2010); Klir & Yuan (1995).